

CSCE 642 DEEP REINFORCEMENT LEARNING · MODULE 05 · LECTURE NOTES

Policy Gradients

Optimise the policy directly.

MODULE OUTCOMES

- Derive the policy gradient theorem.
- Explain why the derivation needs no model of the environment.
- Explain baselines and variance reduction.
- Compare policy gradients against value-based methods.
- Explain why on-policy methods discard their data.

1 The derivation

$$\nabla J(\theta) = E[\nabla \log \pi(a|s;\theta) \cdot R]$$

The gradient of the expected return equals the expectation of the score function times the return. The step that makes it possible is the log-derivative trick: $\nabla p = p \cdot \nabla \log p$, which converts the gradient of an expectation (which you cannot compute, because it involves the environment's distribution) into an expectation of a gradient (which you can estimate by sampling).

Note carefully what is absent from the formula: any gradient of the environment. There is no ∇P and no model at all — the dynamics appear only inside the expectation, which you estimate by running the policy and observing what happens. That absence is the entire reason the method works: you are differentiating only your own policy, which you wrote in a framework that differentiates things. Teaching the derivation by pointing at what is missing is clearer than following the algebra, because the algebra's purpose is precisely to eliminate that term.

REINFORCE, AND WHY IT IS SO HIGH-VARIANCE

Run an episode, compute its total return, and increase the log-probability of every action that was taken, in proportion to that return. That is the whole algorithm, and it is four lines of code.

It is unbiased — the estimate's expectation is the true gradient — and it does work on small problems, which makes it worth implementing once.

The variance is enormous, for two separable reasons. First, the return is a Monte Carlo estimate and carries every random event in the episode (Module 03 §1's table). **Second, every action in the episode is credited with the whole return** — including actions taken *after* the reward was earned, which cannot possibly have caused it.

So the gradient estimate is dominated by which episodes happened to go well rather than by which actions made them go well, which is a credit-assignment failure rather than merely noise. §2 is entirely about fixing it, and both of its main fixes attack one of the two reasons.

2 Variance reduction

SUBTRACT A BASELINE:

$$\text{grad} = E[\text{grad} \log \pi(a|s) * (R - b(s))]$$

THIS DOES NOT CHANGE THE EXPECTATION, for any b that does not depend on the action:

$$\begin{aligned} E[\text{grad} \log \pi(a|s) * b(s)] \\ &= b(s) * \sum_a \pi(a|s) * \text{grad} \log \pi(a|s) \\ &= b(s) * \text{grad} \sum_a \pi(a|s) \\ &= b(s) * \text{grad } 1 \\ &= 0 \end{aligned}$$

The subtracted term has expectation EXACTLY ZERO.

SO THE BIAS IS UNCHANGED AND THE VARIANCE FALLS, frequently by an order of magnitude. It is as close to free as anything in this field gets.

THE BEST SIMPLE BASELINE is $V(s)$, the expected return from that state. Then $(R - V(s))$ is the ADVANTAGE: how much better this action turned out to be than average from here.

-- and now you need a value function after all, which is Module 06's actor-critic.

TWO MORE FIXES:

CAUSALITY / reward-to-go

credit an action only with rewards that arrived AFTER it. An action cannot affect the past, so including earlier rewards adds variance and no signal. Free, always correct, and it is the fix for the SECOND reason in section 1's callout.

NORMALISE the advantages within each batch -- subtract the mean, divide by the standard deviation.

Technically this introduces a bias (the mean is estimated from the same batch); it is universally done; it helps substantially.

The baseline proof is two lines and worth doing once by hand — it is the clearest example available of reducing variance without touching bias, and the reason it works (the probabilities sum to one, so their gradient sums to zero) is the kind of fact that makes the rest of the field's variance-reduction tricks recognisable.

3 Against value-based methods

	Policy gradient	Value-based (DQN)
What it learns	The policy directly.	Q , and acts greedily with respect to it.
Action spaces	Continuous or discrete, natively — the policy outputs a distribution of whatever shape you choose.	Discrete, and small , because the target requires a max over actions (Module 04 §4).
Stochastic policies	Natural, and sometimes optimal — CSCE 625 Module 12 §4's mixed strategies, and necessary under partial observability.	Awkward — a greedy policy is deterministic, and ϵ -greedy is a poor substitute for a learned distribution.
Sample efficiency	Poor — on-policy , so every batch is discarded after one use (see the callout).	Better — a replay buffer reuses each transition many times.
Stability	Better — no bootstrapping leg, so no deadly triad in the pure form.	Worse — all three legs of the triad (Module 04 §1).
Converges to	A local optimum, reliably — it is gradient ascent on a smooth objective (CSCE 669 Module 05).	Possibly nothing , in the function-approximation case.

ON-POLICY METHODS THROW THEIR DATA AWAY

The policy gradient is an expectation under the *current* policy. So experience collected under an older policy is not a valid sample for the current gradient — it answers a different question.

Which means every batch is used once and discarded. An enormous waste compared to a replay buffer that reuses each transition dozens of times, and it is the direct cause of the sample-efficiency row in the table above.

Importance sampling can correct for the mismatch — reweight each sample by the ratio of its action's probability under the new and old policies — and the variance of that correction explodes as the two policies diverge, because the ratios become extreme and a few samples dominate the estimate.

Which is exactly what PPO's clipping manages (Module 06 §3): reuse a batch for several gradient steps, but only while the policies remain close enough for the importance correction to be trustworthy, and refuse to move further. Understanding PPO as 'partial data reuse under a trust constraint' rather than as a clipping heuristic is the thing to take from this callout.

4 In practice

- Use reward-to-go and a learned value baseline. Plain REINFORCE is a teaching tool rather than a method, and skipping both fixes wastes most of the sample budget on variance.

- **Normalise advantages per batch, and normalise observations too** — CSCE 633 Module 02's **feature scaling, which matters at least as much here**, because the observation scale feeds directly into the policy's conditioning (CSCE 669 Module 05 §2).
- **Log policy entropy, every run. A collapsed entropy means the policy has become deterministic and has stopped exploring** — which explains most performance plateaus and is invisible in the return curve until it is too late. Module 07 §3 makes entropy an explicit objective for exactly this reason.
- **Use large batches. The gradient is a noisy expectation and its variance falls as $1/B$** (CSCE 669 Module 08 §2), **so batch size matters considerably more here than in supervised learning** — and the practical consequence is that parallel environment collection is the standard engineering pattern.
- **And watch the gradient norm. Clip it**; spikes are real, they are caused by rare extreme advantages, and a single one can destroy a policy that took hours to train. **Entropy and gradient norm are the two most informative logs**, and **neither is the loss** — which Module 04 §4 established is uninformative.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Sutton & Barto — chapter 13 (free PDF)	The policy gradient theorem and REINFORCE , with the baseline derivation of §2.
OpenAI Spinning Up — Intro to Policy Optimization (free)	The clearest available derivation of §1 , including the reward-to-go and baseline arguments with the algebra shown.
Williams — Simple statistical gradient-following algorithms (free)	The original REINFORCE paper , and it is clearer than its reputation.
Schulman et al. — High-Dimensional Continuous Control Using Generalized Advantage Estimation (free)	§2's variance reduction taken properly — GAE interpolates the bias-variance trade in the advantage, and it is what every modern implementation uses.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Derive the policy gradient** and identify the log-derivative step.
- 2 **Verify the baseline's zero expectation** algebraically.
- 3 **Implement REINFORCE on CartPole** and measure the gradient variance across batches.
- 4 **Add reward-to-go** and report the variance change.
- 5 **Add a learned value baseline** and report it again.

- 6 **Add advantage normalisation** and report a third time.
- 7 **Plot all four variances on one figure** and relate each to its fix.
- 8 **Sweep batch size** from 256 to 8192 and plot performance and gradient variance.
- 9 **Log policy entropy** and deliberately cause it to collapse. Observe the plateau.
- 10 **Implement importance sampling to reuse a batch twice** and measure how the correction's variance grows.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 State the policy gradient theorem and the trick that derives it.
- 2 What is absent from the formula, and why does that matter?
- 3 Give the two reasons REINFORCE is high-variance.
- 4 Prove that a baseline does not change the expectation.
- 5 What is the advantage, and what does using V as a baseline require?
- 6 Why is reward-to-go always correct?
- 7 Compare policy gradients against value-based methods on six axes.
- 8 Why do on-policy methods discard data, and what does PPO do about it?
- 9 Name the two most informative things to log.